



Programming Skills Assessment

Data Structures & Algorithms — Coding Round

Each question carries marks as indicated. Read all questions carefully before starting.

Q7. Move all zeros to end without changing relative order of non-zeros

Given an integer array, move all 0s to the end while maintaining the relative order of non-zero elements. You must do this in-place without making a copy of the array.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$ **Marks:** 1

Test Cases:

#	Input	Expected Output
1	[0, 1, 0, 3, 12]	[1, 3, 12, 0, 0]
2	[0, 0, 0, 1]	[1, 0, 0, 0]
3	[1, 2, 3]	[1, 2, 3]
4	[0]	[0]
5	[0, 0]	[0, 0]

Q8. Check if two strings are anagrams of each other

Given two strings s and t, return true if t is an anagram of s, and false otherwise. An anagram is a word formed by rearranging all the letters of another word, using each original letter exactly once.

Time Complexity: $O(n)$ **Space Complexity:** $O(1)$ **Marks:** 1

Test Cases:

#	Input	Expected Output
1	s="anagram", t="nagaram"	True
2	s="rat", t="car"	False
3	s="a", t="a"	True
4	s="ab", t="a"	False
5	s="listen", t="silent"	True



Q9. Longest substring without repeating characters

Given a string s , find the length of the longest substring without repeating characters. A substring is a contiguous sequence of characters within the string.

Time Complexity: $O(n)$ **Space Complexity:** $O(\min(n,m))$ where m = charset size **Marks: 2**

Test Cases:

#	Input	Expected Output
1	"abcabcbb"	3 ("abc")
2	"bbbbbb"	1 ("b")
3	"pwwkew"	3 ("wke")
4	" "	0
5	"dvdvf"	3 ("vdf")

Q10. Count number of set bits (1s) in an integer's binary representation

Write a function that takes an unsigned integer and returns the number of 1 bits it has (also known as the Hamming weight). Solve using Brian Kernighan's algorithm.

Time Complexity: $O(k)$ where k = number of set bits **Space Complexity:** $O(1)$ **Marks: 2**

Test Cases:

#	Input	Expected Output
1	$n = 11$ (binary: 1011)	3
2	$n = 128$ (binary: 10000000)	1
3	$n = 4294967293$ (binary: 11111111...11101)	31
4	$n = 0$	0
5	$n = 255$ (binary: 11111111)	8

Q11. Check balanced parentheses for {}, [], and ()

Given a string s containing only the characters '(', ')', '{', '}', '[', and ']', determine if the input string is valid. An input string is valid if every open bracket is closed by the same type of bracket and in the correct order.

Time Complexity: $O(n)$ **Space Complexity:** $O(n)$ **Marks: 2**

Test Cases:

#	Input	Expected Output
1	" () "	True
2	" () [] {} "	True
3	" (] "	False
4	" ([] "	False
5	" { [] } "	True

**Q12. Find height of a binary tree**

Given the root of a binary tree, return its height. Height is defined as the number of edges on the longest path from root to a leaf node. An empty tree has height -1 and a single node tree has height 0.

Time Complexity: $O(n)$ **Space Complexity:** $O(h)$ where h = height (recursion stack) **Marks: 2**

Test Cases:

#	Input	Expected Output
1	Tree: [3, 9, 20, null, null, 15, 7]	2
2	Tree: [1, null, 2]	1
3	Tree: [] (empty)	-1
4	Tree: [1] (single node)	0
5	Tree: [1, 2, 3, 4, 5]	2

Grand Total: 10



Pattern Printing Questions

P13. Number border with alphabet fill

Print a square pattern of size n where the border is filled with the row number and the interior is filled concentrically with alphabets — A for the outermost inner ring, B for the next ring inward, and so on.

Marks: 1

Sample output (n = 7):

1	1	1	1	1	1	1
2	A	A	A	A	A	2
3	A	B	B	B	A	3
4	A	B	C	B	A	4
5	A	B	B	B	A	5
6	A	A	A	A	A	6
7	7	7	7	7	7	7

P14. Diamond — * border, alphabets from center outward

Print a diamond shape where the outermost shell is made of '*'. Inside the shell, 'A' sits at the top tip and the center of each row. Letters increase (B, C, D...) as you move outward from the center toward the * border. The pattern is symmetric both horizontally and vertically.

Marks: 1

Sample output (n = 5):

```
      *
    * A *
  * B A B *
* C B A B C *
* D C B A B C D *
  * C B A B C *
    * B A B *
      * A *
      *
```